

Below is the **interview-ready version** of your Node.js + Express + REST prep. I'm also fixing a few weak or inaccurate statements from the source so you do not repeat nonsense in an interview.

0) What interviewers actually want

They do **not** want copied definitions.

They want to see whether you understand:

1. **What it is**
2. **Why it matters**
3. **How it works internally**
4. **When to use it**
5. **What can go wrong**

So for every topic, use this pattern:

Definition → **internal working** → **example** → **one common mistake** → **one interview line**

1) Node.js Fundamentals

1.1 What is Node.js?

What it is

Node.js is a **runtime environment** that lets you run JavaScript outside the browser.

Important correction

JavaScript is the language. Node.js is **not** JavaScript. Node.js is the engine/runtime that executes JavaScript on the server side.

How to say it in interview

“Node.js is a server-side runtime built on Chrome’s V8 engine. It allows JavaScript to run outside the browser, mainly for backend development, APIs, and real-time applications.”

Example

```
console.log("Hello from Node.js");
```

Run:

```
node app.js
```

Common mistake

Saying “Node.js is a language” is wrong.

1.2 Why is Node.js popular?

Core reasons

- Fast for I/O-heavy work
- Non-blocking asynchronous model
- Same language on frontend and backend
- Huge npm ecosystem
- Good for APIs, real-time apps, streaming, microservices

How to say it in interview

“Node.js is popular because it handles many concurrent I/O operations efficiently using an event-driven non-blocking model. That makes it a strong fit for APIs, chat apps, streaming, and backend services.”

Example use cases

- Chat applications
 - Live notifications
 - REST APIs
 - File uploads/downloads
 - Real-time dashboards
-

1.3 Is Node.js single-threaded?

Correct answer

Yes for the main event loop, but not literally only one thread for everything.

Important correction

Node.js uses:

- one main thread for JavaScript execution
- background threads via **libuv thread pool** for some tasks like:
 - file system work
 - crypto
 - compression
 - DNS lookups

How to say it in interview

“Node.js runs JavaScript on a single main thread, but it uses libuv and a thread pool for certain background operations. So it is single-threaded in execution of JavaScript, not single-threaded in the entire runtime.”

Why this matters

This is where many candidates give shallow answers and get exposed.

1.4 Node.js architecture

Simple working model

1. Request comes in
2. Event loop checks the request
3. If task is non-blocking, Node handles it quickly
4. If task is offloadable, libuv takes care of it
5. When done, callback/promise resolution returns to event loop
6. Response goes back

Better interview line

“Node.js uses an event loop to manage asynchronous operations. It does not wait for every request to finish before moving on. Instead, it delegates slow operations and comes back when results are ready.”

Important correction

Not every database query goes to the thread pool. Many DB drivers use non-blocking network I/O. The thread pool is mainly for specific native operations, not your entire backend.

1.5 Event Loop

What it is

The event loop is the mechanism that keeps checking what callback or promise handler should run next.

Very short explanation

It keeps Node.js alive and responsive without blocking the main thread.

Interview answer

“The event loop is the core scheduling mechanism in Node.js. It processes queued callbacks and handles asynchronous work completion without blocking the main JavaScript thread.”

Harder interview follow-up

You may be asked about phases:

- timers
- pending callbacks
- idle/prepare
- poll
- check
- close callbacks

You do **not** need to memorize every phase unless the role is backend-heavy, but you should know:

- setTimeout() → timers phase
- setImmediate() → check phase

2) Synchronous vs Asynchronous

2.1 Synchronous programming

Meaning

One task must finish before the next starts.

Example

```
console.log("A");  
console.log("B");  
console.log("C");
```

Output:

```
A  
B  
C
```

Problem

If one operation is slow, everything waits.

2.2 Asynchronous programming

Meaning

Start a task, do not wait for it to finish, move ahead.

Example

```
console.log("Start");

setTimeout(() => {
  console.log("Async done");
}, 2000);
```

```
console.log("End");
```

Output:

```
Start
End
Async done
```

Interview line

“Asynchronous programming allows Node.js to handle other work while waiting for I/O operations to complete.”

2.3 Callbacks

What they are

A callback is a function passed into another function to run later.

Example

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

greet("Ashu", () => {
  console.log("Callback executed");
});
```

Problem

Deep nesting becomes ugly.

2.4 Callback Hell

Meaning

Nested callbacks become hard to read, debug, and maintain.

Example

```
loginUser(() => {  
  getProfile(() => {  
    getPosts(() => {  
      getComments(() => {  
        console.log("Done");  
      });  
    });  
  });  
});
```

Fix

Use:

- Promises
- async/await
- modular functions

How to say it

“Callback hell is deep nesting of asynchronous callbacks. It makes code hard to read and maintain, so we avoid it using Promises or async/await.”

2.5 Promises

Meaning

A Promise represents a future result: success or failure.

Example

```
const p = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Data loaded"), 1000);  
});  
  
p.then(console.log).catch(console.error);
```

Interview line

“Promises improve async code structure by avoiding nested callbacks and giving clear success/failure handling.”

2.6 Async/Await

Meaning

Syntactic sugar over Promises. Makes async code look synchronous.

Example

```
async function loadData() {  
  try {  
    const result = await Promise.resolve("Hello");  
    console.log(result);  
  } catch (err) {  
    console.error(err);  
  }  
}  
loadData();
```

Interview line

“Async/await is built on Promises and makes asynchronous code easier to read and handle with try/catch.”

Common mistake

Thinking await works outside async function in normal code. It does not.

3) Modules, package.json, npm

3.1 What is a module?

Meaning

A module is a reusable file or block of code.

Example

math.js

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = { add };
```

app.js

```
const { add } = require("./math");  
console.log(add(2, 3));
```

Interview line

“Modules help split code into reusable units and keep the codebase clean.”

3.2 CommonJS vs ES Modules

This is important and often asked.

CommonJS

```
const x = require("./x");  
module.exports = something;
```

ES Modules

```
import x from "./x.js";  
export default something;
```

Interview line

“Node.js traditionally used CommonJS, but ES Modules are now also supported. CommonJS uses `require` and `module.exports`, while ESM uses `import` and `export`.”

3.3 What is package.json?

Meaning

It stores project metadata, scripts, dependencies, and version info.

Example

```
{  
  "name": "my-app",  
  "version": "1.0.0",  
  "scripts": {  
    "start": "node app.js",  
    "dev": "nodemon app.js"  
  }  
}
```


How to create it

```
npm init -y
```

Interview line

“package.json defines project metadata, scripts, and dependency information. It is the central config file of a Node project.”

3.4 What is npm?

Meaning

npm is the package manager for Node.js.

Common commands

```
npm install express  
npm install  
npm uninstall express  
npm run dev  
npm start
```

package-lock.json

Important because it locks exact dependency versions for reproducible installs.

Interview line

“npm manages third-party packages. package-lock.json ensures consistent dependency versions across environments.”

4) Built-in Node.js modules

These are common interview topics.

4.1 http

Used to create servers.

```
const http = require("http");  
  
const server = http.createServer((req, res) => {  
  res.end("Hello");  
});
```

```
server.listen(3000);
```

4.2 fs

Used for file operations.

```
const fs = require("fs");

fs.readFile("test.txt", "utf8", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

4.3 path

Used for file paths.

```
const path = require("path");
console.log(path.join(__dirname, "files", "a.txt"));
```

4.4 os

System info.

```
const os = require("os");
console.log(os.cpus().length);
```

4.5 events

Used for event-based programming.

5) Events and EventEmitter

What it is

Event-driven programming means actions happen in response to events.

Example

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("orderPlaced", (orderId) => {
  console.log("Order received:", orderId);
});

emitter.emit("orderPlaced", 101);
```

Interview line

“EventEmitter lets us register listeners with on and trigger events using emit. It is useful for decoupled event-driven architecture.”

Why it matters

Used in:

- logging
 - notifications
 - custom events
 - streaming pipelines
-

6) Buffer, Streams, Timers

6.1 Buffer

Meaning

Buffer stores binary data.

Why it matters

Useful for files, images, network packets.

Example

```
const buf = Buffer.from("Hello");  
console.log(buf);
```

Interview line

“Buffer is Node’s way of handling raw binary data efficiently.”

6.2 Streams

Meaning

Streams process data in chunks instead of loading everything at once.

Types

- Readable
- Writable
- Duplex

- Transform

Example

```
const fs = require("fs");
const readStream = fs.createReadStream("bigfile.txt");

readStream.on("data", (chunk) => {
  console.log(chunk.toString());
});
```

Why streams matter

They reduce memory usage for large files.

Interview line

“Streams are used for efficient handling of large data by processing it chunk by chunk.”

6.3 setTimeout vs setImmediate

Correct explanation

- setTimeout(fn, 0) runs after a timer delay, but not instantly
- setImmediate(fn) runs in the **check** phase after I/O callbacks

Important correction

Do **not** say setImmediate has “priority 0”. That is not a correct interview answer.

Interview line

“setTimeout schedules execution after a timer delay, while setImmediate runs after the poll phase of the event loop, often useful after I/O.”

Example

```
setTimeout(() => console.log("timeout"), 0);
setImmediate(() => console.log("immediate"));
```

Order can vary depending on context.

7) Express.js

7.1 What is Express?

Meaning

Express is a web framework built on Node.js that simplifies route handling, middleware, and server logic.

Interview line

“Node.js gives the runtime, Express gives the web framework layer for routing, middleware, and request/response handling.”

Setup commands

```
npm init -y
npm install express
npm install nodemon --save-dev
```

Basic server

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello Express");
});

app.listen(3000, () => console.log("Server running on 3000"));
```

7.2 Routing

Meaning

Routing decides which code runs for a specific URL and HTTP method.

Example

```
app.get("/users", (req, res) => {
  res.send("Get all users");
});

app.post("/users", (req, res) => {
  res.send("Create user");
});
```

Interview line

“Routing maps HTTP methods and URL paths to handler functions.”

7.3 Middleware

Meaning

Middleware sits between request and response.

Uses

- logging
- authentication
- validation
- parsing
- rate limiting

Example

```
app.use((req, res, next) => {  
  console.log(req.method, req.url);  
  next();  
});
```

Very important

If you forget next(), the request can hang.

Interview line

“Middleware is a function that can access req, res, and next. It is used to process or intercept requests before they reach the route handler.”

7.4 Types of middleware

Common ones

- Application middleware: app.use()
- Router middleware
- Built-in middleware: express.json()
- Error-handling middleware
- Third-party middleware

Example of error middleware

```
app.use((err, req, res, next) => {  
  console.error(err);  
  res.status(500).send("Something went wrong");  
});
```

8) REST and HTTP

8.1 What is REST?

Meaning

REST is an architectural style for designing APIs around resources.

How to say it

“REST is a style for designing APIs using resource-based URLs and standard HTTP methods.”

Example resources

- /users
 - /orders
 - /products/12
-

8.2 RESTful API

A RESTful API follows REST principles.

Important correction

REST does **not** mean only JSON. JSON is common, but REST can use other formats too.

8.3 HTTP methods

GET

Fetch data.

GET /users

POST

Create new resource.

POST /users

PUT

Replace full resource.

PUT /users/12

PATCH

Partial update.

PATCH /users/12

DELETE

Remove resource.

DELETE /users/12

Interview line

“GET reads, POST creates, PUT replaces, PATCH partially updates, and DELETE removes.”

Important mistake to avoid

Saying PUT and PATCH are the same. They are not.

8.4 Idempotency

Meaning

Same request repeated multiple times gives same final result.

Idempotent methods

- GET
- PUT
- DELETE

Non-idempotent

- POST

Example

Deleting the same user twice should still leave it deleted.

Interview line

“Idempotency means repeating the same request does not change the outcome after the first successful execution.”

8.5 Status codes

You should know more than just 200 and 500.

Must-know ones

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 422 Unprocessable Entity
- 500 Internal Server Error

Interview line

“Status codes communicate request outcome, whether success, client error, authentication issue, not found, or server failure.”

8.6 CORS

Meaning

Browser security rule controlling cross-origin requests.

Important correction

CORS is mainly enforced by **browsers**, not by the backend itself.

Example

Frontend running on localhost:3000 calling backend on localhost:5000 may need CORS enabled.

Express command

```
npm install cors
```

Usage

```
const cors = require("cors");  
app.use(cors());
```

Interview line

“CORS is a browser-based security policy that restricts cross-origin requests unless the server explicitly allows them.”

8.7 API versioning

Meaning

Maintain multiple API versions like /api/v1 and /api/v2.

Why

To avoid breaking old clients.

Interview line

“API versioning helps maintain backward compatibility while evolving the API.”

9) Authentication and JWT

9.1 What is JWT?

Meaning

JWT is a signed token used for stateless authentication.

Important correction

JWT is usually **signed**, not encrypted. People often confuse this.

The payload can be decoded; integrity is protected by signature.

Flow

1. User logs in
2. Server generates JWT
3. Client stores it
4. Client sends it in header on future requests
5. Server verifies signature

Example header

Authorization: Bearer <token>

Interview line

“JWT is a signed token used for stateless authentication. The server verifies the signature to trust the token.”

9.2 What is wrong with many JWT answers?

Bad answer:

“JWT keeps data secret.”

That is not correct.

Better answer:

“JWT proves authenticity and integrity. It does not automatically hide the payload unless additional encryption is used.”

9.3 Common auth-related interview topics you should also know

Sessions vs JWT

- Sessions: server stores session data
- JWT: client stores token, server verifies signature

Password hashing

Use bcrypt, not plain text.

npm install bcrypt

Example

```
const bcrypt = require("bcrypt");
const hash = await bcrypt.hash("mypassword", 10);
```

Security basics

- bcrypt for passwords
 - httpOnly cookies
 - token expiry
 - refresh tokens
 - rate limiting
 - input validation
-

10) Commands you should know

Project setup

```
node -v
npm -v
npm init -y
npm install express
npm install nodemon --save-dev
```

Run app

```
node app.js
npm run dev
```

package.json scripts

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```

Common installs

```
npm install cors
npm install dotenv
npm install helmet
npm install morgan
npm install bcrypt
npm install jsonwebtoken
```

11) Important topics missing from your source but very likely to be asked

This part matters. These are often the difference between average and strong interviews.

11.1 process and environment variables

What to know

- process.env
- process.argv
- process.exit()

Example

```
console.log(process.env.PORT);
```

Use dotenv

```
npm install dotenv
```

```
require("dotenv").config();
```

Interview line

“Environment variables keep sensitive config out of source code.”

11.2 Error handling

Basic rule

Never ignore errors.

Example

```
try {  
  const data = JSON.parse("bad");  
} catch (err) {  
  console.error("Invalid JSON");  
}
```

Express error handling

Use centralized middleware.

Interview line

“Good backend code handles errors consistently and returns meaningful HTTP status codes.”

11.3 File uploads and body parsing

Know these

```
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

Interview line

“Body parsers convert incoming request bodies into usable JavaScript objects.”

11.4 Security basics

You should know:

- helmet
- cors
- rate limiting
- sanitization
- password hashing
- HTTP-only cookies
- validation

Example

npm install helmet express-rate-limit

Why interviewers ask

They want to know if you can build real production code, not just toy APIs.

11.5 Logging

Use:

- morgan
- Winston / Pino

Example

npm install morgan

Interview line

“Logging is essential for debugging, observability, and production support.”

11.6 Database connection pooling

This is very important for backend interviews.

What it means

Instead of opening a new DB connection for every request, reuse connections from a pool.

Why

Better performance and stability.

Interview line

“Connection pooling improves performance by reusing database connections instead of creating them repeatedly.”

11.7 Scaling Node.js

Know these terms

- clustering
- load balancing
- horizontal scaling
- vertical scaling
- caching
- queues

Important correction

Node is not magically enough for everything. CPU-heavy tasks can block the event loop.

Interview line

“For CPU-intensive work, Node.js is not ideal on the main thread. We use worker threads, child processes, queues, or move that work to another service.”

11.8 Worker threads vs child process

Worker threads

Good for CPU-heavy JS tasks.

Child process

Run separate processes for heavy or isolated tasks.

Interview line

“Worker threads share memory more efficiently, while child processes give stronger isolation.”

12) Hard interview questions and how to learn them

These are the ones that separate a strong candidate from someone who memorized definitions.

12.1 Hard question: Why is Node.js good for I/O-heavy apps but not CPU-heavy tasks?

Good answer

Because the event loop should stay free. CPU-heavy tasks block the main thread and delay all requests.

How to learn it

Build a test:

- one route with file read or database call
 - one route with an expensive loop
- Compare latency.
-

12.2 Hard question: What happens in the event loop when async code runs?

Good answer

The main thread schedules async work, returns immediately, and later the callback or promise handler gets executed when the operation completes.

How to learn it

Read about:

- event loop phases
 - microtasks vs macrotasks
 - promises vs timers
-

12.3 Hard question: Why is await not just a callback replacement?

Good answer

await is syntax built on Promises. It improves readability, but it still depends on the Promise system underneath.

How to learn it

Convert callback code → Promise code → async/await code.

12.4 Hard question: Difference between PUT and PATCH

Good answer

PUT usually replaces the whole resource, PATCH updates only selected fields.

How to learn it

Use a sample user object and update:

- name only with PATCH
 - complete object with PUT
-

12.5 Hard question: JWT vs session

Good answer

Sessions store state on server. JWT is stateless and stores claims in token form, signed by server.

How to learn it

Draw flow diagrams for both.

13) Interview-ready answers you can memorize

These are clean, direct answers.

Node.js

“Node.js is a server-side JavaScript runtime built on V8. It uses an event-driven non-blocking model to handle concurrent I/O efficiently.”

Express

“Express is a lightweight Node.js framework that simplifies routing, middleware, and HTTP request handling.”

Middleware

“Middleware is a function that processes the request-response cycle and can modify req, res, or pass control using next().”

REST

“REST is an architectural style that organizes APIs around resources and uses HTTP methods like GET, POST, PUT, PATCH, and DELETE.”

CORS

“CORS is a browser security policy that restricts cross-origin requests unless the server allows them.”

JWT

“JWT is a signed token used for stateless authentication. The server verifies the token signature on each request.”

14) What to say if they ask you to build something

If interviewer says: “Design a REST API for users”

Say:

1. Define resource: /users
2. Use GET, POST, PUT/PATCH, DELETE
3. Validate input
4. Return proper status codes
5. Secure with JWT
6. Handle errors centrally
7. Add logging and CORS if needed

That answer sounds practical and mature.

15) Brutally honest warning about interview mistakes

Do **not** say these wrong things:

- “Node.js is a language”
- “Node.js is completely single-threaded”
- “JWT encrypts data”
- “PUT and PATCH are the same”
- “CORS is a server bug”
- “Async means multithreading”
- “setImmediate has priority 0”
- “DB queries always go to thread pool”

Those answers will make you look unprepared.

16) Best way to study this fast

Do this in order:

1. Node.js basics
 2. Event loop + async programming
 3. Modules + npm + package.json
 4. Express routing + middleware
 5. REST + HTTP methods + status codes
 6. JWT + auth basics
 7. Streams + buffer
 8. Security + error handling
 9. Scaling + database pooling
 10. Practice building 2–3 APIs
-

17) Mini practice list

You should be able to answer these without freezing:

- What is Node.js?
 - Why is Node.js fast?
 - What is the event loop?
 - What is the difference between callback, promise, and async/await?
 - What is middleware?
 - What is routing?
 - What is CORS?
 - What is idempotency?
 - Difference between PUT and PATCH?
 - What is JWT?
 - What is the difference between session and JWT?
 - What are streams?
 - What is Buffer?
 - Why use package.json?
 - What is npm?
 - What is error-handling middleware?
-

I can turn this into a **clean interview question bank with model answers**, one topic at a time, so you can revise it like notes instead of reading a wall of text.